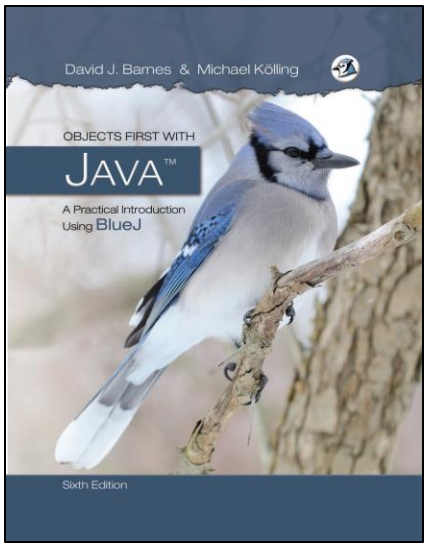
Unit 7

Manejo de errores



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson del capítulo 14 del libro: “Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition” escrito por David J. Barnes y Michael Kolling. Pearson Education, Inc. © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Programación “defensiva”.
  - Anticipar posibles errores.
- Lanzamiento y manejo de excepciones.
- Informar de errores.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Situaciones típicas de error

- Implementación incorrecta.
  - No cumple la especificación.
- Acceso a objetos incorrecto.
  - P.ej. índice no válido.
- Estados de objetos inconsistente o no apropiado.
  - P. ej. mediante extensión de una clase.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

No siempre son errores de programación

- Los errores a veces provienen del entorno:
  - URL introducida incorrectamente.
  - Corte en la red.
- El procesamiento de ficheros es particularmente propenso a errores:
  - Archivos perdidos.
  - Falta de permisos de acceso.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

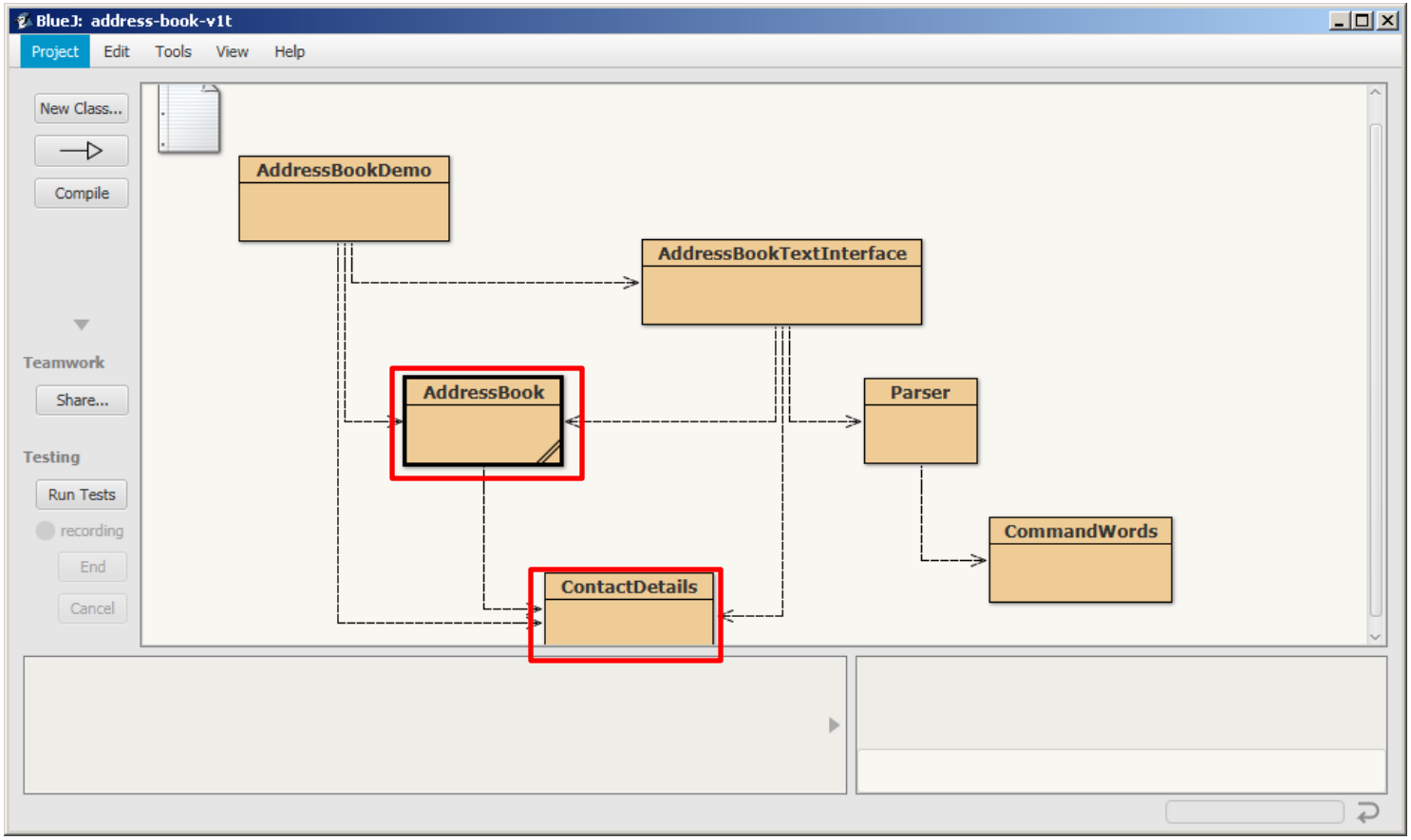
Explorando errores

- Exploramos situaciones de error en el Proyecto **address-book**.
- Dos aspectos:
  - Aviso de error.
  - Tratamiento de error.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase AddressBook



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase AddressBook

```
public class AddressBook
{
    // Storage for an arbitrary number of details.
    private TreeMap<String, ContactDetails> book;
    private int numberOfEntries;

    /**
     * Perform any initialization for the address book.
     */
    public AddressBook()
    {
        book = new TreeMap<>();
        numberOfEntries = 0;
    }

    /**
     * Look up a name or phone number and return the
     * corresponding contact details.
     * @param key The name or number to be looked up.
     * @return The details corresponding to the key.
     */
    public ContactDetails getDetails(String key)
    {
        return book.get(key);
    }
```

```
    /**
     * Return whether or not the current key is in use.
     * @param key The name or number to be looked up.
     * @return true if the key is in use, false otherwise.
     */
    public boolean keyInUse(String key)
    {
        return book.containsKey(key);
    }

    /**
     * Add a new set of details to the address book.
     * @param details The details to associate with the person.
     */
    public void addDetails(ContactDetails details)
    {
        book.put(details.getName(), details);
        book.put(details.getPhone(), details);
        numberOfEntries++;
    }

    /**
     * Change the details previously stored under the given key.
     * @param oldKey One of the keys used to store the details.
     * @param details The replacement details.
     */
    public void changeDetails(String oldKey,
                              ContactDetails details)
    {
        removeDetails(oldKey);
        addDetails(details);
    }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase AddressBook

```java
/**
 * Search for all details stored under a key that starts with
 * the given prefix.
 * @param keyPrefix The key prefix to search on.
 * @return An array of those details that have been found.
 */
public ContactDetails[] search(String keyPrefix)
{
    // Build a list of the matches.
    List<ContactDetails> matches = new LinkedList<>();
    // Find keys that are equal-to or greater-than the prefix.
    SortedMap<String, ContactDetails> tail = book.tailMap(keyPrefix);
    Iterator<String> it = tail.keySet().iterator();
    // Stop when we find a mismatch.
    boolean endOfSearch = false;
    while(!endOfSearch && it.hasNext()) {
        String key = it.next();
        if(key.startsWith(keyPrefix)) {
            matches.add(book.get(key));
        }
        else {
            // As the list is sorted, there won't be any more.
            endOfSearch = true;
        }
    }
    ContactDetails[] results = new ContactDetails[matches.size()];
    matches.toArray(results);
    return results;
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase AddressBook

```
/**
 * Return the number of entries currently in the
 * address book.
 * @return The number of entries.
 */
public int getNumberOfEntries()
{
    return numberOfEntries;
}

/**
 * Remove an entry with the given key from the address book.
 * @param key One of the keys of the entry to be removed.
 */
public void removeDetails(String key)
{
    ContactDetails details = book.get(key);
    book.remove(details.getName());
    book.remove(details.getPhone());
    numberOfEntries--;
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase AddressBook

```
/**
 * Return all the contact details, sorted according
 * to the sort order of the ContactDetails class.
 * @return A sorted list of the details.
 */
public String listDetails()
{
    // Because each entry is stored under two keys, it is
    // necessary to build a set of the ContactDetails. This
    // eliminates duplicates.
    StringBuilder allEntries = new StringBuilder();
    Set<ContactDetails> sortedDetails = new TreeSet<>(book.values());
    for(ContactDetails details : sortedDetails) {
        allEntries.append(details).append("\n\n");
    }
    return allEntries.toString();
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase ContactDetails

```
public class ContactDetails implements Comparable<ContactDetails>
{
    private String name;
    private String phone;
    private String address;

    /**
     * Set up the contact details. All details are trimmed to remove
     * trailing white space.
     * @param name The name.
     * @param phone The phone number.
     * @param address The address.
     */
    public ContactDetails(String name, String phone, String address)
    {
        // Use blank strings if any of the arguments is null.
        if(name == null) {
            name = "";
        }
        if(phone == null) {
            phone = "";
        }
        if(address == null) {
            address = "";
        }
        this.name = name.trim();
        this.phone = phone.trim();
        this.address = address.trim();
    }
```

```
    /**
     * @return The name.
     */
    public String getName()
    {
        return name;
    }

    /**
     * @return The telephone number.
     */
    public String getPhone()
    {
        return phone;
    }

    /**
     * @return The address.
     */
    public String getAddress()
    {
        return address;
    }
```

Pearson Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase ContactDetails

```
/**
 * Test for content equality between two objects.
 * @param other The object to compare to this one.
 * @return true if the argument object is a set
 *              of contact details with matching attributes.
 */
public boolean equals(Object other)
{
    if(other instanceof ContactDetails) {
        ContactDetails otherDetails = (ContactDetails) other;
        return name.equals(otherDetails.getName()) &&
               phone.equals(otherDetails.getPhone()) &&
                address.equals(otherDetails.getAddress());
    }
    else {
        return false;
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase ContactDetails

```
/**
 * Compare these details against another set, for the purpose
 * of sorting. The fields are sorted by name, phone, and address.
 * @param otherDetails The details to be compared against.
 * @return a negative integer if this comes before the parameter,
 *         zero if they are equal and a positive integer if this
 *         comes after the second.
 */
public int compareTo(ContactDetails otherDetails)
{
    int comparison = name.compareTo(otherDetails.getName());
    if(comparison != 0){
        return comparison;
    }
    comparison = phone.compareTo(otherDetails.getPhone());
    if(comparison != 0){
        return comparison;
    }
    return address.compareTo(otherDetails.getAddress());
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t -> Clase ContactDetails

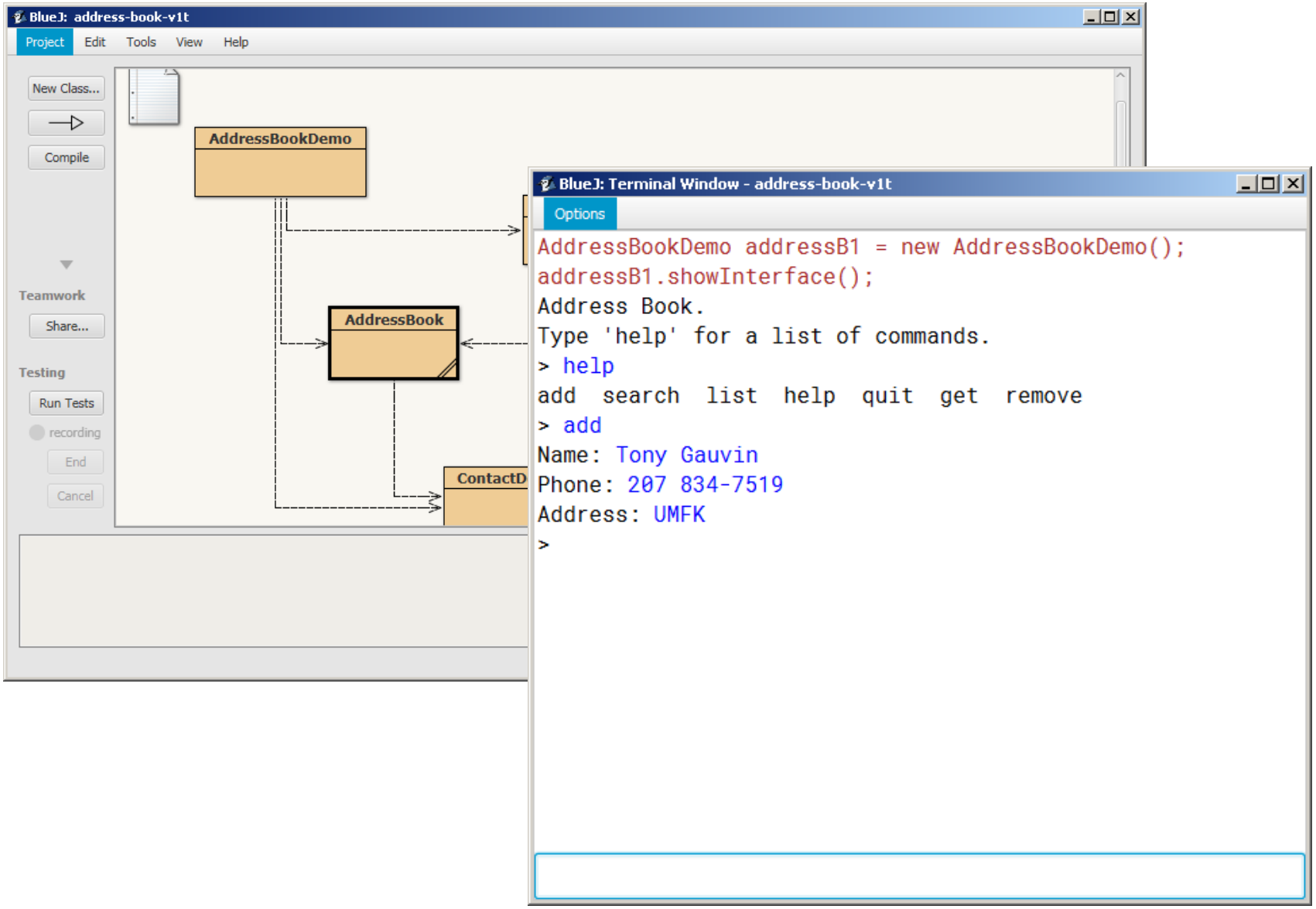
```java
/**
 * @return A multi-line string containing the name,
 * phone, and address.
 */
public String toString()
{
    return name + "\n" + phone + "\n" + address;
}

/**
 * Compute a hashcode using the rules to be found in
 * "Effective Java", by Joshua Bloch.
 * @return A hashcode for ContactDetails.
 */
public int hashCode()
{
    int code = 17;
    code = 37 * code + name.hashCode();
    code = 37 * code + phone.hashCode();
    code = 37 * code + address.hashCode();
    return code;
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Adress-book-v1t



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Programación “defensiva”

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Programación “defensiva”

- Interacción cliente-servidor.
  - ¿Debería asumir el servidor que los clientes son fiables?
  - ¿O debería asumir que los clientes son “enemigos” potenciales?
- Se requieren diferencias significativas en la implementación.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Problemas que debemos abordar

- ¿Cuánto debe controlar el servidor las llamadas a métodos?
- ¿Cómo se puede informar de los errores?
- ¿Cómo puede un cliente anticipar el fracaso?
- ¿Cómo debe un cliente enfrentarse al fracaso?

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo

La clase **AddressBook** asume que los clientes son fiables

- Creamos un objeto **`AddressBook`**.
- Intentamos borrar una entrada.
- Se produce un error en tiempo de ejecución.
  - **¿De quién es la "culpa"?**
- La anticipación y la prevención son preferibles a culpar a nadie.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros

- Los parámetros representan una mayor 'vulnerabilidad' para un objeto de tipo servidor.
  - Los parámetros en los constructores inicializan el estado.
  - Los parámetros en los métodos a menudo son responsables de su comportamiento.
- **La comprobación de parámetros es una medida defensiva**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros

La clase **AddressBook** asume que la *key* recibida es una clave válida para la agenda

si el mapa no tiene esa *key*, la variable **details** contendrá **null**

```
public void removeDetails(String key)
    {
        ContactDetails details = book.get(key);
        book.remove(details.getName());
        book.remove(details.getPhone());
        numberOfEntries--;
    }
```

Eso no es un error;

pero el error aparece en la siguiente instrucción, donde asumimos que *details* hace referencia a un objeto válido:

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros – comprobando la clave

```
public void removeDetails(String key)
{
    if(keyInUse(key)) {
        ContactDetails details = book.get(key);
        book.remove(details.getName());
        book.remove(details.getPhone());
        numberOfEntries--;
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros

Otros ejemplos donde la clase **AddressBook** assume que los clientes son fiables

¿¿Qué ocurre si es null??

```
public void addDetails(ContactDetails details)
    {
        book.put(details.getName(), details);
        book.put(details.getPhone(), details);
        numberOfEntries++;
    }
```

Deberíamos solucionarlo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros

Otros ejemplos donde la clase **AddressBook** assume que los clientes son fiables

¿¿Qué ocurre si son null??

```
public void changeDetails(String oldKey,
                          ContactDetails details)
    {
        removeDetails(oldKey);
        addDetails(details);
    }
```

Deberíamos solucionarlo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Valores de parámetros

Otros ejemplos donde la clase **AddressBook** assume que los clientes son fiables

¿¿Qué ocurre si es null??

```
public ContactDetails[] search(String keyPrefix)
    {
      // Build a list of the matches.
      List<ContactDetails> matches = new LinkedList<>();
        // Find keys that are equal-to or greater-than
        //the prefix.
        SortedMap<String, ContactDetails> tail =
                    book.tailMap(keyPrefix);
        …
    }
```

Deberíamos solucionarlo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Informando de errores desde el servidor

- ¿A quién debemos informar sobre errores de parámetros erróneos?
  - ¿Al usuario?
    - ¿Es un usuario humano?
    - ¿Puede resolver el problema?
  - ¿Al objeto cliente?
    - Devolvemos un valor de control.
    - **Lanzamos una excepción**.

Sólo se usa en circunstancias limitadas

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Devolviendo un valor de control

Antes void -> ahora boolean



```
public boolean removeDetails(String key)
{
    if(keyInUse(key)) {
        ContactDetails details = book.get(key);
        book.remove(details.getName());
        book.remove(details.getPhone());
        numberOfEntries--;
        return true;
    }
    else {
        return false;
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El cliente puede comprobar el valor devuelto

```
if(contacts.removeDetails("…")) {
    // Entry successfully removed.
    // Continue as normal.
    …
}
else {
    // The removal failed.
    // Attempt a recovery, if possible.
    …
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Devolviendo valor de control – otro ejemplo

El mismo tipo que antes

```
public ContactDetails getDetails(String key)
{
    if(keyInUse(key)) {
        return book.get(key)
}
    else {
        return null;
    }
}
```

Pero ahora se devuelve *null* si la clave no se está utilizando

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El cliente puede comprobar el valor devuelto – otro ejemplo

```
ContactDetails details=contacts.getDetails("David");
if(details != null) {
    // Send a text message to David.
    // Continue as normal.
    …
}
else {
    // Failed to find the entry.
    // Attempt a recovery, if possible.
    …
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Respuestas potenciales de clientes

- Comprobar el valor devuelto.
  - Intenta la recuperación en caso de error.
  - Evita fallos de programa.
- Ignorar el valor de retorno.
  - No se puede prevenir.
  - Es probable que provoque un fallo de programa.

- **Son preferibles las 'Excepciones'**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Respuestas potenciales de clientes

- Una solicitud fallida no representará un error de programación lógica en muchos casos
- Una solicitud incorrecta casi seguro representa un error lógico de programa.
- Deberíamos esperar acciones de seguimiento de cliente bastante diferentes en las distintas situaciones.
  - Lamentablemente, no existe una **forma general** de resolver el conflicto **utilizando simplemente valores de retorno**.
- **Son preferibles las 'Excepciones'**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Lanzamiento y manejo de excepciones

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Lanzamiento de excepciones - Principios

- Es un **característica** especial **del lenguaje**.
- No se necesita ningún valor de retorno 'especial'.
  - Es independiente del valor de retorno de un método
  - Puede ser utilizado por todos los métodos, independientemente del tipo de retorno.
- Los errores no pueden ser ignorados en el cliente.
  - Se **interrumpe** el flujo de control normal.
- Se recomiendan acciones de recuperación específicas.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Lanzamiento de excepciones - Principios

- Punto importante a tener en cuenta:
  - Cuando hay excepciones, el sitio donde se descubre un error será distinto del lugar donde se intenta la recuperación (si corresponde) :
    - **se descubrirá en** el método del **servidor**,
    - **se recuperará en** el **cliente**

Si la recuperación fuera posible en el momento del descubrimiento, ¡entonces no tendría sentido lanzar una excepción!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones – Ejemplo: *ArrayIndexOutOfBoundsException*

```
1   [...]
2   void arrayMagic(){
3      int[] array  =   new int[2];
4      for (int i = 0; i< array.length; i ++    )       {
5          array[i] = 0;
6      }
7      System.out.println(array[2]);
8   }
9   [...]
```

```
> _ Exception in thread "main"
  java.lang.ArrayIndexOutOfBoundsException: 2
  at ExceptionExamples.arrayMagic(ExceptionExamples.java:  7)
  (...)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones – Ejemplo: *Type Mismatch*

```
1  [...]
2  void type(){
3    int  i = "5";
4  }
5  [...]
```

```
>_  Exception in thread "main" java.lang.Error: Unresolved
    compilation problem:
    Type mismatch: cannot convert from String to int
    at ExceptionExamples.type (...)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones – Ejemplo: *Variables sin un valor por defecto*

```
1   [...]
2   void  testVariables(){
3           String test;
4           System.out.println(test);
5   }
6   [...]
```

```
> Exception in thread "main" java.lang.Error:
  Unresolved compilation problem:
  The local variable name may not have been initialized
  (...)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones – Ejemplo: *Operaciones con null (objeto no existente)*

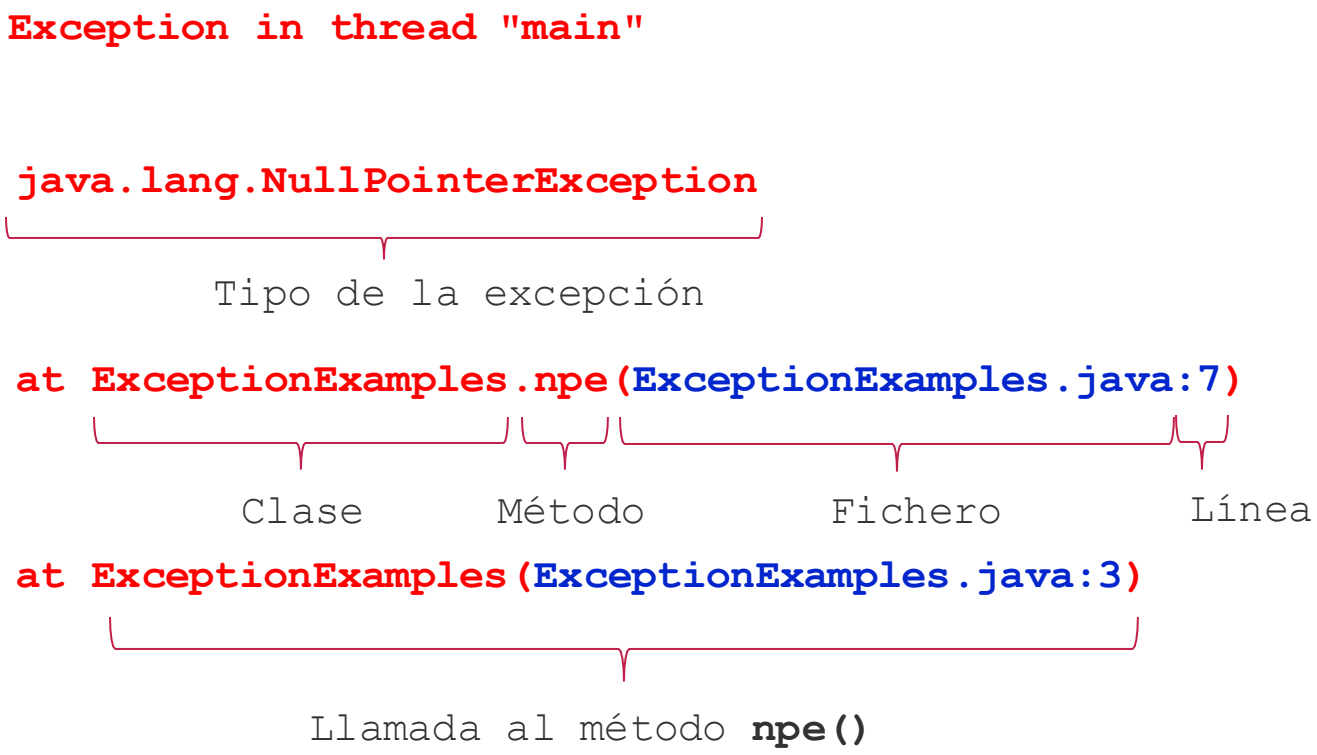
```
1  class ExceptionExamples{
2     public static void main(String args[]){
3     npe();
4     }
5     static void npe(){
6         Detective duke =   new  Detective();
7         System.out.println(duke.getName().toLowerCase());
8
9     }
   [...]
```

```
> Exception in thread "main"
  java.lang.NullPointerException at
  ExceptionExamples.nullPointerException
  (ExceptionExamples.java:7)
  at ExceptionExamples(ExceptionExamples.java:3)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Descomponiendo una excepción – Ejemplo: *Operaciones con null (objeto no existente)*



Pearson

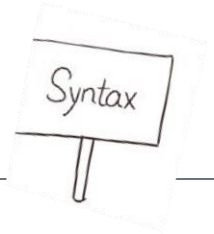
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones – Definión y sintaxis



- Es un “evento excepcional”
- Es un evento que interrumpe el flujo normal de un programa cuando ocurre un error
- Es un objeto

```
try {
  //  podría lanzarse una excepción
} catch (<<ExceptionType>> e) {
  // recuperación desde la excepción
}
```



- El tipo de excepción que se va a detectar va entre paréntesis (p.ej. *IOException*, *ArrayIndexOutOfBoundsException*, ...)
- Es una forma de permitir que el programa maneje la excepción (sin terminar el programa)

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Lanzando una excepción (1 of 2)

```
/**
 * Look up a name or phone number and return the
 * corresponding contact details.
 * @param key The name or number to be looked up.
 * @return The details corresponding to the key,
 *         or null if there are none matching.
 * @throws IllegalArgumentException if the key is invalid.
 */
public ContactDetails getDetails(String key)
{
    if(key == null) {
        throw new IllegalArgumentException(
                        "null key in getDetails");
    }
    return book.get(key);
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Lanzando una excepción (2 of 2)

- Sintaxis:

  throw new *ExceptionType*(**"*optional-diagnostic-string*"**);

  - Primero se crea un objeto de tipo excepción:
    - `new ExceptionType("...")`
  - Después se lanza el objeto excepción:
    - `throw ...`

- Documentación Javadoc:
  - `@throws ExceptionType reason`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El efecto de una excepción

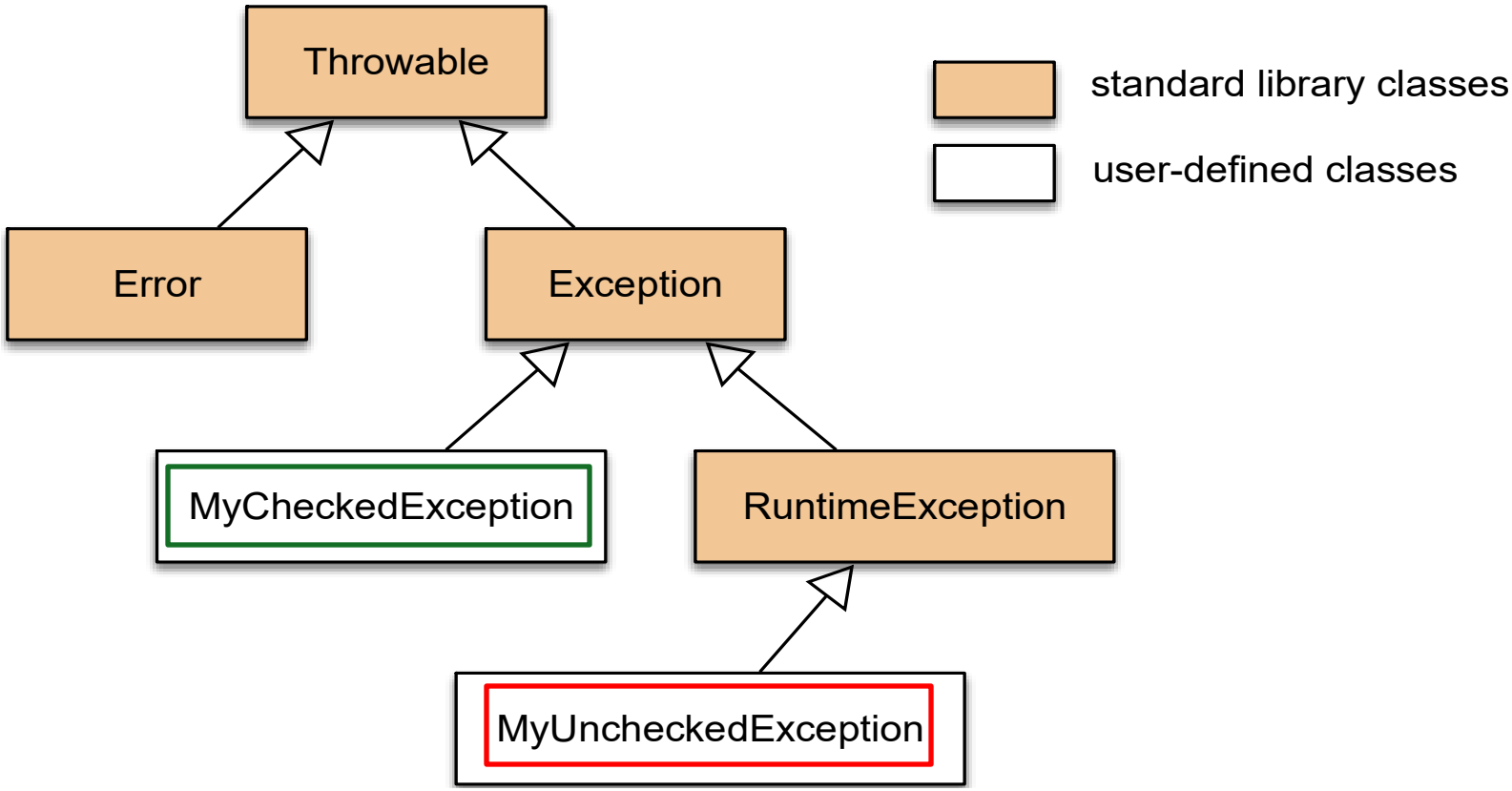
- El método que la lanza termina antes de tiempo.
  - No se devuelve ningún valor.
    - Incluso si el método tiene un tipo de retorno no nulo
    - El compilador indica un error si se escriben sentencias después de una sentencia *throw*, porque nunca se ejecutarían.
- El control no vuelve al punto de llamada del cliente.
  - Por lo que el cliente no puede continuar.

- El **cliente puede capturar (*catch*) una excepción**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

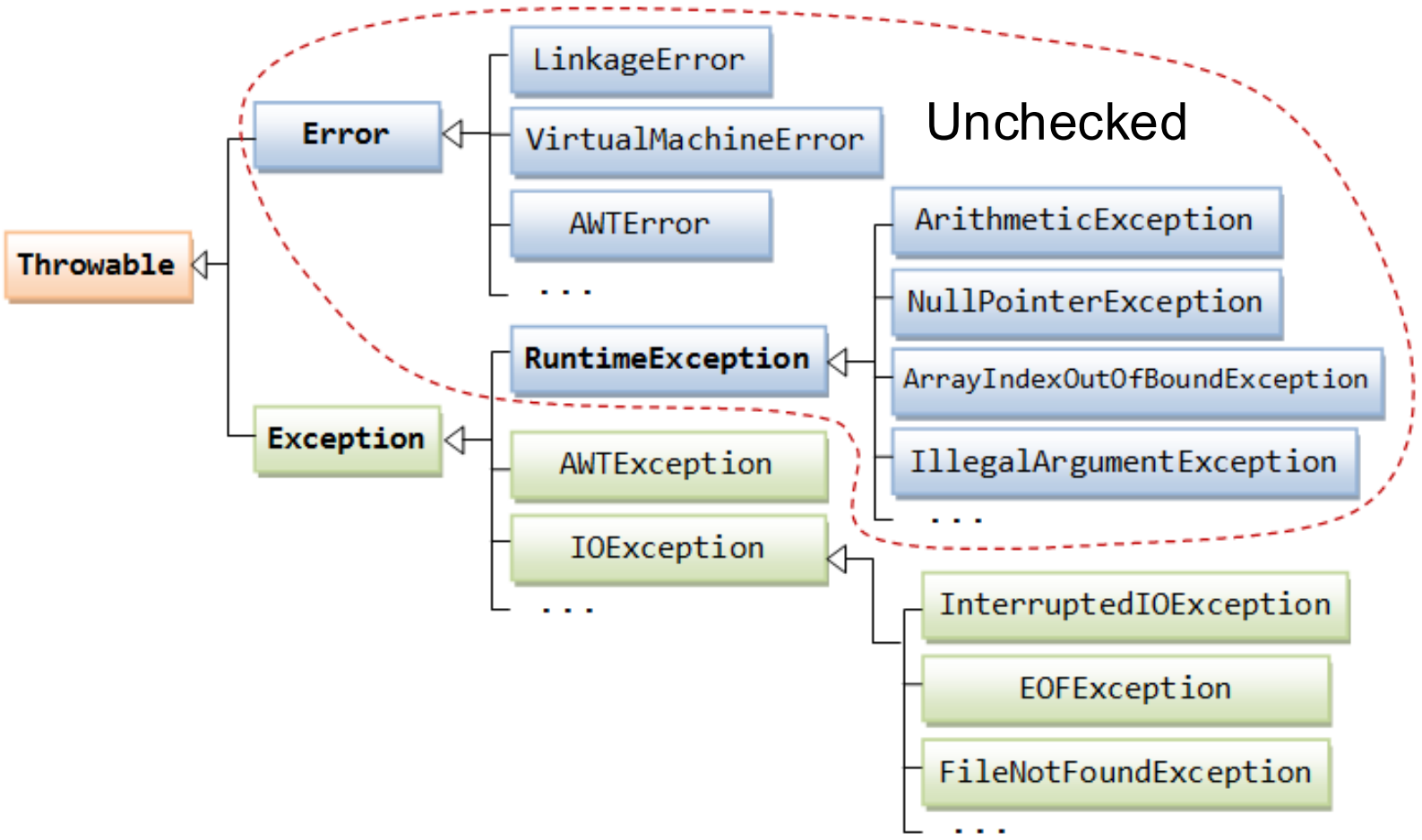
La jerarquía de clases *Exception*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La jerarquía de *excepciones*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Categorías de excepciones

- Excepciones comprobadas
  - Subclase of **`Exception`**
  - Usadas para **anticipar fallos que podemos manejar**.
    - situaciones que escapan al control del programador/a
      - p.ej. se llena un disco al intentar escribir en un fichero
  - La recuperación es posible.
- Excepciones no comprobadas
  - Subclase of **`RuntimeException`**
  - Usadas para **fallos de programación no anticipados**.
    - Situaciones que podrían evitarse
      - p.ej. llamar a un método pasando una variable a **null**
  - La recuperación es poco probable

    -> **Fallo de aplicación**.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Excepciones no comprobadas

- El compilador ‘no las comprueba’.
- Provocar la terminación del programa si no se detecta.
  - Es la práctica normal.
- **`IllegalArgumentException`** es un ejemplo típico.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Comprobando parámetros

```
public ContactDetails getDetails(String key)
{
    if(key == null) {
        throw new IllegalArgumentException(
                        "null key in getDetails");
    }
    if(key.trim().length() == 0) {
        throw new IllegalArgumentException(
                    "Empty key passed to getDetails");
    }
    return book.get(key);
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Prevenir en la creación de objetos

```
public ContactDetails(String name, String phone, String address)
{
    if(name == null) {
        name = "";
    }
    if(phone == null) {
        phone = "";
    }
    if(address == null) {
        address = "";
    }

    this.name = name.trim();
    this.phone = phone.trim();
    this.address = address.trim();

    if(this.name.length() == 0 && this.phone.length() == 0) {
        throw new IllegalStateException(
                    "Either the name or phone must not be blank.");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Manejo de excepciones

- **Las excepciones comprobadas** deben capturarse y responderse.
  - debe haber instrucciones catch y/o throws

¡Distinto de *throw*!

- El compilador asegura que su uso está controlado.
  - En los objetos servidor y cliente.
- Si se usa correctamente, los errores pueden recuperarse.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Instrucción *throws*

- Los métodos que lanzan una excepción comprobada:
  - deben incluir la instrucción throws en su cabecera:

```
public void saveToFile(String File) throws IOException {
...
}
```

- Está permitido usar una cláusula throws para excepciones no controladas,
  - pero el compilador no lo requiere.
- Recomendamos que una cláusula throws **solo se utilice** para listar las excepciones controladas que lanza un método.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La sentencia *try* (1 of 2)

- Los clientes capturan una excepción comprobada:
  - se protege la llamada con una sentencia try:

```
try {
    Proteger una o más sentencias aquí.
}
catch(Exception e) {
    Informar y recuperarse de la excepción
     aquí.
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La sentencia *try* (2 of 2)

1. La excepción se lanza aquí

```
String filename = null;
Boolean successful;
try {
     filename = request-a-file-from-the-user;
     addressbook.saveToFile(filename);
     successful = true;
}
catch(IOException e) {
     System.out.println("Unable to save to " + filename);
     successful = false;
}
...
```

Instrucciones protegidas

2. El control se transfiere aquí

Si no surge ninguna excepción durante la ejecución de las *instrucciones protegidas*, el bloque *catch* se saltará cuando finalice el bloque *try*.
La ejecución continuará en la línea siguiente a la instrucción *try/catch*.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Capturando excepciones múltiples

```
try {
    ...
    ref.process();
    ...
}
catch(EOFException e) {
    // Actúa sobre una excepción end-of-file.
    ...
}
catch(FileNotFoundException e) {
    // Actúa sobre una excepción file-not-found.
    ...
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Multi-catch

```
try {
    ...
    ref.process();
    ...
}
catch(EOFException | FileNotFoundException e) {
    // Actúa sobre los dos tipos de excepciones.
    ...
}
```

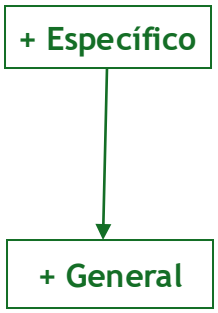
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“catch” polimórfico

(incorrecto)

```
try{
        …
        some.process();
        …
}
catch(Exception e) {
        doRecovery();
}
catch (EOFException e)
{
        doFileRecovery();
}
```



(correcto)

```
try{
        …
        some.process();
        …
}
catch(EOFException e) {
        doFileRecovery();
}
catch (Exception e) {
        doRecovery();
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La cláusula *finally* (1 of 2)

```
try {
      Proteger una o más sentencias aquí.
}
catch(Exception e) {
     Informar y recuperarse de la excepción aquí.
}
finally {
 Realizar cualquier acción aquí tanto si se lanza
    una excepción o no.
}
```

A menudo se omite

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La cláusula *finally* (2 of 2)

- ¿Qué diferencia hay con este código?

```
try {
      Proteger una o más sentencias aquí.
}
catch(Exception e) {
    Informar y recuperarse de la excepción aquí.
}
Realizar cualquier acción aquí tanto si se lanza
   una excepción o no.
```

- Una cláusula finally se ejecuta incluso si se ejecuta un return en las cláusulas try o catch.
- Si se lanza una excepción en el bloque try pero no se captura, la cláusula finally se sigue ejecutando.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Definiendo nuevas excepciones (excepciones propias) (1 of 4)

- Usadas cuando las clases de excepción estándar no describen satisfactoriamente el error producido
- Heredamos (*extend*) de `RuntimeException` para una excepción no controlada o de `Exception` para una controlada.
- Definimos nuevos tipos para dar más información.
  - Incluímos información de informe y/o recuperación.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Definiendo nuevas excepciones (excepciones propias) (2 of 4)

```
public class NoMatchingDetailsException extends Exception
{
    private String key;

    public NoMatchingDetailsException(String key)
    {
        this.key = key;
    }

    public String getKey()
    {
        return key;
    }

    public String toString()
    {
        return "No details matching '" + key + "' were found.";
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Definiendo nuevas excepciones (excepciones propias) (3 of 4)

```
public void removeDetails(String key)
       throws NoMatchingDetailsException
    {
        if(key == null){
            throw new IllegalArgumentException(
                        "Null key passed to removeDetails.");
        }
        if(keyInUse(key)) {
            ContactDetails details = book.get(key);
            book.remove(details.getName());
            book.remove(details.getPhone());
            numberOfEntries--;
        }
        else {
            throw new NoMatchingDetailsException(key);
        }
    }
```

Si no hay entradas en la agenda que coincidan con *key*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Definiendo nuevas excepciones (excepciones propias) (4 of 4)

```
public void changeDetails(String oldKey,
                          ContactDetails details)
    {
        if(details == null) {
            throw new IllegalArgumentException(
                    "Null details passed to changeDetails.");
        }
        if(oldKey == null){
            throw new IllegalArgumentException(
                    "Null key passed to changeDetails.");
        }
        if(keyInUse(oldKey)){
            try {
                removeDetails(oldKey);
                addDetails(details);
            }
            catch(NoMatchingDetailsException e) {
                System.out.println(e);
            }
        }
    }
```

Necesita modificarse porque llama a *removeDetails*() que lanza una excepción *controlada*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Recuperación de errores

- Los clientes deben prestar atención a las notificationes de error.
  - **Comprobar los valores de retorno**.
  - **No 'ignorar' las excepciones**.

```
AddressDetails details = null;
try {
      details = contacts.getDetails(...);
}
catch(Exception e) {
      System.out.println("Error: " + e);
}
String phone = details.getPhone();
```

¿¿¿Qué ocurre si getDetails() lanza una excepción???

- Incluir código para intentar la recuperación.
  - A menudo necesitamos un bucle.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Intento de recuperación

```
// Try to save the address book.
boolean successful = false;
int attempts = 0;
do {
    try {
        contacts.saveToFile(filename);
        successful = true;
    }
    catch(IOException e) {
        System.out.println("Unable to save to " + filename);
        attempts++;
        if(attempts < MAX_ATTEMPTS) {
            filename = an alternative file name;
        }
    }
} while(!successful && attempts < MAX_ATTEMPTS);

if(!successful) {
    Report the problem and give up;
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Intento de recuperación

- Anticipar un error e intentar recuperarlo, por lo general, requerirá un flujo de control más complejo que si el error no pudiera ocurrir.
- Las sentencias del bloque *catch* son clave para configurar el intento de recuperación.
- La recuperación a menudo implicará volver a intentarlo.
- No se puede garantizar el éxito de la recuperación.
- Debería existir algún camino para seguir intentando una recuperación imposible.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Evitar el error

- Los clientes pueden utilizar métodos de consulta del servidor para evitar errores.
  - Clientes más robustos implica que los servidores pueden ser más confiados.
  - Se pueden utilizar excepciones no controladas (pero de forma limitada).
  - Simplifica la lógica del cliente.
- Puede incrementar el acoplamiento cliente-servidor.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otra bibliografía:

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Exceptions (Oracle tutorials):
  - Exceptions
  - Class Exception
  - Catching and Handling Exceptions

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved